



Caviar Private Pools Audit Report

Prepared by: X3 Security

Table of Contents

- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
- [High](#)

Protocol Summary

Caviar Private Pools is an NFT Automated Market Maker (AMM) platform that allows users to create highly customizable, owner-controlled liquidity pools. Each private pool supports features like concentrated liquidity, customizable fee rates, stolen NFT filtering, royalty support, and flash loans. Liquidity providers can deposit both NFTs and ETH into these pools, enabling seamless trading and liquidity provisioning for NFTs.

Core Components:

1. **Factory Contract:** The Factory contract enables the creation and initialization of new custom pools, using a proxy system that points to a reference implementation. Each pool is represented by an NFT that signifies ownership. All protocol fees are accrued by the factory and can be withdrawn by the protocol admin. While the initial protocol fee rate is set to 0%, it may be increased with prior notice.
2. **PrivatePool Contract:** This contract contains the core functionality for managing custom pools. It allows pool owners to define custom settings such as liquidity concentration, fee structures, NFT weightings, royalty fee support, and more. It also supports flash loans and filtering for stolen NFTs. Traders can utilize these pools to buy, sell, or swap NFTs within the pool.
3. **EthRouter Contract:** The EthRouter facilitates multi-pool transactions. For instance, if a user wants to buy multiple NFTs across different pools, the EthRouter can handle the sequence of actions and execute them in a single transaction. This contract integrates with both private and public Caviar pools, streamlining complex NFT trades.
4. **PrivatePoolMetadata Contract:** This contract is responsible for generating on-chain metadata and SVG representations of the NFTs that signify ownership of a custom pool. These metadata and visual representations are used for displaying NFTs across marketplaces and wallets.

Audit Period: April 07, 2023 - April 13, 2023

Disclaimer

The X3 Security team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

Scope

```
contracts/
├── src/
│   ├── Factory.sol
│   ├── PrivatePoolMetadata.sol
│   ├── EthRouter.sol
│   └── PrivatePool.sol
└── interfaces/
    └── IStolenNftOracle.sol
```

Executive Summary

Issues found

Findings

Severity	ID	Title
High	H-01	[H-01] Consider using OpenZeppelin's SafeCast library to prevent unexpected overflows when casting from uint256

High

[H-01] Consider using OpenZeppelin's SafeCast library to prevent unexpected overflows when casting from

uint256

Severity: Low Risk

Likelihood: Low

Relevant GitHub Links:

[Link to buy function in PrivatePool.sol](#)

[Link to sell function in PrivatePool.sol](#)

Summary

In the `PrivatePool.sol` contract, the `buy()` and `sell()` functions first declare the variables `netInputAmount`, `feeAmount`, `protocolFeeAmount`, and `weightSum` as `uint256`. Later in these functions, these variables are cast to `uint128` and used to update the virtual reserves. This downcasting is done without checking if the values fit within the `uint128` range, leading to a risk of silent overflows.

Vulnerability Details

The `buy()` and `sell()` functions update the `virtualBaseTokenReserves` and `virtualNftReserves` using values that are initially declared as `uint256`. These values are then cast to `uint128` before updating the reserves. However, this casting is performed without validating that the values fit into a `uint128` type, creating a potential for silent overflows. If an overflow occurs, it could disrupt the pool's internal logic, particularly the `xy=k` invariant, which would result in incorrect pricing and financial losses.

The relevant code snippets from the `buy()` and `sell()` functions are as follows:

```
// From the buy() function
virtualBaseTokenReserves += uint128(netInputAmount - feeAmount -
protocolFeeAmount);
virtualNftReserves -= uint128(weightSum);

// From the sell() function
virtualBaseTokenReserves -= uint128(netOutputAmount + protocolFeeAmount +
feeAmount);
virtualNftReserves += uint128(weightSum);
```

Impact

- **Silent Overflow:** The lack of checks before casting could cause a silent overflow, leading to the breakdown of the `xy=k` equation, which underpins the pricing logic of the AMM.
- **Incorrect Price Calculation:** This would result in inaccurate prices and could cause significant financial losses for traders and liquidity providers.
- **Pool Instability:** Overflow issues could destabilize the pool, resulting in unexpected behavior, incorrect virtual reserve updates, and inaccurate value representation.

Proof of Concept (PoC)

The following proof of concept demonstrates the risk of overflow with a high-decimal token:

```

function test_Overflow() public {
    // Setup pool and base token with high decimal value (e.g., 30 decimals)
    HighDecimalsToken baseToken = new HighDecimalsToken();
    privatePool = new PrivatePool(address(factory), address(royaltyRegistry),
address(stolenNftOracle));
    privatePool.initialize(
        address(baseToken),
        nft,
        100_000_000 * 1e30,
        10 * 1e18,
        changeFee,
        feeRate,
        merkleRoot,
        true,
        false
    );

    // Mint NFTs and setup pool
    for (uint256 i = 100; i < 110; i++) {
        nft.mint(address(privatePool), i);
    }
    for (uint256 i = 100; i < 108; i++) {
        tokenIds.push(i);
    }

    // Save K constant value before the trade
    uint256 kBefore = uint256(privatePool.virtualBaseTokenReserves()) *
uint256(privatePool.virtualNftReserves());

    // Execute the buy function
    (uint256 netInputAmount,, uint256 protocolFeeAmount) = privatePool.buyQuote(8
* 1e18);
    deal(address(baseToken), address(this), netInputAmount);
    baseToken.approve(address(privatePool), netInputAmount);
    privatePool.buy(tokenIds, tokenWeights, proof);

    // Save K constant value after the trade and assert it remains unchanged
    uint256 kAfter = uint256(privatePool.virtualBaseTokenReserves()) *
uint256(privatePool.virtualNftReserves());
    assertEq(kBefore, kAfter, "K constant changed due to overflow");
}

contract HighDecimalsToken is ERC20 {
    constructor() ERC20("High Decimals Token", "HDT", 30) {}
}

```

Tools Used

- Manual Review

Recommendations

It is recommended to use OpenZeppelin's [SafeCast](#) library to safely cast from `uint256` to `uint128` and avoid

potential overflows. Add validation checks to ensure that the values being cast do not exceed the `uint128` maximum value. For example:

```
// In the buy function
if (netInputAmount - feeAmount - protocolFeeAmount > type(uint128).max) revert
Overflow();
virtualBaseTokenReserves += uint128(netInputAmount - feeAmount -
protocolFeeAmount);
if (weightSum > type(uint128).max) revert Overflow();
virtualNftReserves -= uint128(weightSum);

// In the sell function
if (netOutputAmount + protocolFeeAmount + feeAmount > type(uint128).max) revert
Overflow();
virtualBaseTokenReserves -= uint128(netOutputAmount + protocolFeeAmount +
feeAmount);
if (weightSum > type(uint128).max) revert Overflow();
virtualNftReserves += uint128(weightSum);
```
